Exploring Package.json File

In Angular 18

Exploring Package.json File

1. package.json is a **metadata and dependency configuration file** used by **Node.js** projects, including Angular applications.

Key Purposes of package.json:

Purpose	Description
1. Dependency Management	Lists all **npm packages** (like Angular, RxJS, etc.) your project depends on.
2. Version Control	Specifies **which versions** of libraries your app uses (e.g., Angular 18, RxJS 7.x).
3. Scripts	Defines **custom npm commands** like ng serve, ng build, npm start, etc.
4. Project Metadata	Stores project info like name, version, description, author, etc.
5. Dev vs Prod Dependencies	Separates dependencies (for runtime) and devDependencies (for build/test tools).

Example: package.json from an Angular app

```
{
 "name": "my-angular-app",
 "version": "1.0.0",
 "scripts": {
  "ng": "ng",
  "start": "ng serve",
  "build": "ng build",
  "test": "ng test"
 },
 "dependencies": {
  "@angular/core": "^18.0.0",
  "rxjs": "^7.5.0"
 },
 "devDependencies": {
  "@angular/cli": "^18.0.0",
  "typescript": "~5.4.0"
 }
}
```

Why is it important?

- You can **reproduce the project setup** by just running npm install – it installs all listed packages.
- Maintains **consistency across developers/teams**.
- Helps CI/CD pipelines install the correct tools automatically.
- Ensures **version control** to avoid breaking changes.

What happens when you run npm install?

- Reads package.json and installs all listed packages into the node_modules/ folder.
- Creates/updates package-lock.json (records exact installed versions for consistency).

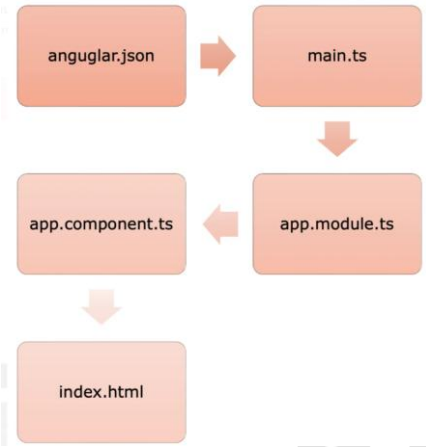
Sai Reddy
saireddy-dotnetfs

2. What happens when you run ng build --prod (or now ng build --configuration production)

This command **builds your application** using the production configuration, which usually includes:

- **AOT (Ahead-of-Time)** compilation.
- **Minification** of JS/CSS.
- **Tree shaking** (removes unused code).
- **Uglification** (shortens variable names).
- **Build optimization**.

The result is a **set of static files** in the dist/your-app-name/ folder.



3. What files are generated in the dist/ folder and why?

Here are the common files and what they do:

File Name	Purpose
main.[hash].js	Contains the **root Angular app module and logic** (your actual application code).
polyfills.[hash].js	Includes **browser compatibility code** (like for older IE or Edge).
runtime.[hash].js	Handles **runtime logic** like module loading, especially for lazy-loaded modules.
scripts.[hash].js	If you included global scripts in angular.json (like jQuery), they go here.
styles.[hash].css	All global styles (from styles.css or SCSS) bundled together.
index.html	The entry point to the app – Angular injects all JS and CSS files here.
favicon.ico, assets/	Static files and assets used in the app.

The [hash] part helps in **cache busting**—when you deploy a new version, the filename changes so the browser fetches the latest version./* in place of [hash] – map will come in angular 13 */

4. Why do we need all these separate files?

Each file has a specific role, improving:

- **Performance**: Smaller files load faster.
- **Code splitting**: Only load parts of app when needed.
- **Browser caching**: If polyfills.js doesn't change, the browser caches it.
- **Modularity**: Keeps logic, styles, and runtime separate for maintainability.

5. What happens when you deploy to Azure or AWS S3?

When you deploy the dist/ folder:

On Azure App Service:

- You can upload the contents of the dist/ folder.
- Azure will serve the index.html file.

- The browser loads the referenced JS/CSS files (like main.js, polyfills.js, etc.).
- Angular bootstraps and runs in the browser (Single Page Application - SPA).

On AWS S3 (Static Website Hosting):

- Upload all dist/ files to the S3 bucket.
- Enable **Static website hosting**.
- Set index.html as the entry point.
- When a user accesses the site, S3 serves index.html, and the browser loads other resources.

Important: For SPAs on S3, configure a custom 404 error page as index.html to handle Angular routes (so routing works correctly even if the user hits a deep link like /dashboard).